

COMP1201 Algorithmic

Revision 2023

Hikmat Farhat

Dynamic programming

- Given a weed control fabric of length 7 meters and the prices for different lengths as below, use an appropriate algorithmic strategy to determine the maximum revenue obtainable by cutting up the weed control fabric and selling the pieces. Explain your working.

Length (meters)	1	2	3	4	5	6	7
Price (GBP)	2	5	9	10	12	13	17

Unbounded knapsack

- This problem is similar to the knapsack problem.
- It is called the unbounded knapsack problem.
- The only difference between it and the 0-1 knapsack is that an item can be selected multiple times
- For the 0-1 knapsack we had

$$opt[i][w] = \max \begin{cases} opt[i-1][w] & \text{if item } i \text{ is not selected} \\ p_i + opt[i-1][w - w_i] & \text{if item } i \text{ is selected} \end{cases}$$

- For this case we only have to change the case when the item is selected.

Note i instead of $i - 1$ on line 9

```
1 Knapsack( $w, v, W$ )
   /* if  $W = 0$ ,  $\text{opt} = 0$  for any  $n$  */
2 for  $i = 1$  to  $n$  do
3   |  $S[i][0] = 0$ 
   /* for any  $W$ , zero items gives  $\text{opt} = 0$  */
4 for  $i = 1$  to  $W$  do
5   |  $S[0][i] = 0$ 
   /* Now the solution */
6 for  $j = 1$  to  $W$  do
7   | for  $i = 1$  to  $n$  do
8     | | if  $j \geq w[i]$  then
9       | | |  $S[i][j] = \max(S[i - 1][j], v[i] + S[i][j - w[i]])$ 
10      | | else
11      | | |  $S[i][j] = S[i - 1][j]$ 
```

Solution

- Note the modification to "walk backwards" to obtain the solution.
- The code for this problem can be found [here](#)

i/w	0	1	2	3	4	5	6	7
{}	0	0	0	0	0	0	0	0
{1}	0	2	4	6	8	10	12	14
{1,2}	0	2	5	7	10	12	15	17
{1,2,3}	0	2	5	9	11	14	18	20
{1,2,3,4}	0	2	5	9	11	14	18	20
{1,2,3,4,5}	0	2	5	9	11	14	18	20
{1,2,3,4,5,6}	0	2	5	9	11	14	18	20
{1,2,3,4,5,6,7}	0	2	5	9	11	14	18	20

Linear programming

- For each of the problems below state whether they are LP problems or not.

$$\begin{aligned} &\text{maximise: } 23x + 22yz - 43xz, \\ &\text{subject to: } 11x + 6y - 19z \leq 17 \\ &\quad \quad \quad x - 11y + 6z \leq 0 \\ &\quad \quad \quad x - z \leq 15 \\ &\quad \quad \quad x, y, z \geq 0. \end{aligned}$$

$$\begin{aligned} &\text{maximise: } 27x + 18y - 31z, \\ &\text{subject to: } 11x - 4xy + 25z \leq 29 \\ &\quad \quad \quad x - 14y + 5z \leq 0 \\ &\quad \quad \quad x - y \leq 15 \\ &\quad \quad \quad x, y, z \geq 0. \end{aligned}$$

Find the maximum of $x + y$ with the following constraints:

$$2x + y \leq 12,$$

$$x + 2y \leq 9,$$

$$x \geq 0,$$

$$y \geq 0.$$

- The first step is to introduce slack variables u and v to convert the inequalities into equalities
- Then the problem is put in tabular form as shown in the next slide

Solution

z	x	x	u	v	
1	-1	-1	0	0	0
	2	1	1	0	12
	1	2	0	1	9

We choose x as the entering variable. The leaving variable is determined by the ratio test. In this case it is u .

z	x	y	u	v	
1	0	-0.5	0.5	0	6
	2	1	1	0	12
	0	1.5	-0.5	1	3

Solution

For convenience, you could (but not necessary) normalize the $(2, 1, 1, 0, 12)$ row

z	x	y	u	v	
1	0	-0.5	0.5	0	6
	1	0.5	0.5	0	6
	0	1.5	-0.5	1	3

The only variable with negative coefficient in the expression for z is y so it enters. Ratio test selects v as the leaving variable.

Solution

z	x	y	u	v	
1	0	0	$1/3$	$1/3$	7
	1	0	$2/3$	$-1/3$	5
	0	1.5	-0.5	1	3

All coefficients are positive so we stop and read the equations from the table

$$z = 7 - 1/3u - 1/3v$$

$$x = 5 - 2/3u + 1/3v$$

$$y = 2 + 1/3u - 2/3v$$

Setting the non-basic variables to 0 we get the optimal values:
 $z = 7, x = 5, y = 2$

Simulated annealing

Simulated annealing is

- a a greedy search algorithm (No unless temperature is 0)
- b deterministic (No. Uses a random numbers to decide)
- c stochastic (Yes. opposite to the above)
- d a neighbourhood search algorithm (Yes. The potential candidate is "close" to the current state)

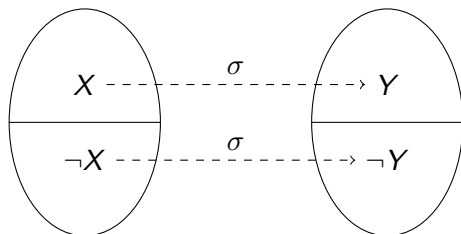
backtracking

Consider the bounded knapsack problem, where an item cannot be used more than 2 times, with n items and a knapsack of size W . A backtracking algorithm A is used to find the optimal solution.

Which statements are true:

- a A visits the state-space tree in a breadth first manner (False).
- b A visits the state-space tree in a depth first manner (True).
- c If there are multiple optimal solutions, A can find all of them (True).
- d When there is no solution, A visits at most $O(2^n)$. (False. The branching factor is 3 so $O(3^n)$)

- Show that if $Y \leq_p X$ and $X \in P$ then $Y \in P$.
- First recall polynomial time reductions.
- $Y \leq_p X$ means $\exists \sigma$ such that $v \in Y \Leftrightarrow \sigma(v) \in X$



Reductions

What we know

- $X \in P$ then there exists a polynomial time algorithm B that decides X . This means that for any u , $B(u) = \text{True} \Leftrightarrow u \in X$.
- $Y \leq_p X$ then there exists a polynomial time function σ such that $v \in Y \Leftrightarrow \sigma(v) \in X$.

Using the above we construct an algorithm to decide Y in polynomial time

- 1 Let v be any instance. Compute $\sigma(v) = u$ in polynomial time.
- 2 Compute $B(u)$ in polynomial.
- 3 If $B(u) = \text{true}$ then $\sigma(v) = u \in X$ and therefore $v \in Y$.
Return true
- 4 If $B(u) = \text{false}$ then $\sigma(v) \notin X$ and therefore $v \notin Y$. Return false
- 5 The above is a procedure to decide in polynomial time any instance v and thus $Y \in P$.

- Consider the Boolean formula. (Note: sometimes we use $\neg x$ and sometimes $\bar{x}$ whichever is more convenient)

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_1) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

- 1 The value of x_1 is deduced (False.)
- 2 The value of x_3 is deduced (True).
- 3 DPLL will backtrack on x_1 (True).
- 4 DPLL will backtrack on all variables (False).

Note the difference between "will" and "might"

At each step the algorithm works as follows:

- 1 The two rules of DPLL are applied to reduce the formula.
- 2 The variables in the reduced formula are guessed in "index order", i.e., x_1 then $x_2, \dots$ etc
- 3 The resulting state-space tree is shown below

